やさしい Zig

JW Lee

やさしい **Zig**

Zig プログラミング言語の包括的な入門ガイド

著者：*JW Lee*

はじめに

やさしい **Zig** へようこそ！この本は、Zig プログラミング言語をゼロから学びたい初心者のために設計されています。Python、JavaScript、C から来た方でも、これが初めてのプログラミング言語であっても、この本がステップバイステップでガイドします。

Zig とは？

Zig は、2016 年に Andrew Kelley によって作成された現代的なシステムプログラミング言語です。「より良い C」として設計されており、C や C++ を困難でエラーが発生しやすくする多くの落とし穴を避けながら、メモリとハードウェアの低レベル制御を提供します。

```
┌──────────────────────────────────────────────────────────┐
│                プログラミング言語スペクトラム                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  高レベル                                    低レベル      │
│  (簡単)                                      (高速)       │
│                                                          │
│  Python ─── JavaScript ─── Go ─── Zig ─── C ─── ASM      │
│     │           │           │       │      │      │       │
│ スクリプト    Webアプリ   サービス  システム  OS   ハードウェア │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

なぜ **Zig** を選ぶのか？

1. シンプルさ

C++ とは異なり、Zig には隠れた制御フロー、隠れたメモリ割り当て、演算子オーバーロードがありません。見たままが得られるものです。

2. ガベージコレクションなしの安全性

Zig はコンパイル時に多くのバグをキャッチします。デフォルトで null ポインタがなく、配列の境界チェック、明示的なエラー処理があります。

3. パフォーマンス

Zig は C や C++ に匹敵する高度に最適化されたマシンコードにコンパイルされます。Uber などの企業で本番環境で使用されています。

4. C 互換性

Zig は C ヘッダーファイルを直接インポートして使用できます。バインディングは不要です。これにより、数十年分の C ライブラリにアクセスできます。

5. Comptime（コンパイル時実行）

Zig はコンパイル時にコードを実行でき、マクロやテンプレートなしで強力なメタプログラミングが可能です。

```
┌──────────────────────────────────────────────────────────┐
│                      Zigの主要機能                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│                                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐            │
│   │ GCなし   │   │ C互換    │   │ Comptime │            │
│   │──────────│   │──────────│   │──────────│            │
│   │手動      │   │.hファイル│   │コンパイル│            │
│   │メモリ    │   │を直接    │   │時にコード│            │
│   │制御      │   │インポート│   │を実行    │            │
│   └──────────┘   └──────────┘   └──────────┘            │
│                                                          │
│                                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐            │
│   │安全性    │   │高速      │   │クロスプラット│         │
│   │──────────│   │──────────│   │──────────│            │
│   │コンパイル│   │LLVM      │   │任意の    │            │
│   │時        │   │最適化    │   │ターゲットに│          │
│   │チェック  │   │コード    │   │コンパイル│            │
│   └──────────┘   └──────────┘   └──────────┘            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Zig のセットアップ

インストール

macOS (Homebrew):

```
brew install zig
```

Linux:

```
# ziglang.org からダウンロードして展開
wget https://ziglang.org/download/zig-linux-x86_64.tar.xz
```

```
tar -xf zig-linux-x86_64.tar.xz
export PATH=$PATH:./zig-linux-x86_64
```

Windows: https://ziglang.org/download/ からインストーラーをダウンロード

インストールの確認

```
zig version
```

最初のコマンド

```
# hello.zig というファイルを作成し、次を実行：
zig run hello.zig
```

この本の使い方

各章では 1 つの概念を次のように紹介します：1. 説明 - 概念とは何か、なぜ重要か 2. 図解 - 物事がどのように機能するかの視覚的表現 3. コード例 - 自分で実行できる動作するコード 4. 出力 - コードを実行したときに表示されるもの 5. キーポイント - 重要なポイントのまとめ

Zig の旅を始めましょう！

第 1 章：Hello World

すべてのプログラミングの旅は Hello World から始まります。このシンプルなプログラムで、Zig プログラムの基本構造を学びます。

プログラム構造の理解

```
┌──────────────────────────────────────────────────────────┐
│                    Zigプログラム構造                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────────────┐                       │
│   │ 1. インポート                 │                       │
│   │    const std = @import("std");│                      │
│   └──────────────────────────────┘                       │
│                   │                                      │
│                   ▼                                      │
│   ┌──────────────────────────────┐                       │
│   │ 2. 定数とグローバル           │                       │
│   │    const MAX = 100;           │                      │
│   └──────────────────────────────┘                       │
│                   │                                      │
│                   ▼                                      │
│   ┌──────────────────────────────┐                       │
│   │ 3. 関数                       │                      │
│   │    fn helper() { ... }        │                      │
```

```
│    └──────────────────────────────────┘                    │
│                      │                                      │
│                      ▼                                      │
│    ┌──────────────────────────────────┐                    │
│    │  4. MAIN関数（エントリーポイント）    │                    │
│    │     pub fn main() void { ... }     │                    │
│    └──────────────────────────────────┘                    │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

最初のプログラム

```zig
// ============================================================
// 第 1 章：Hello World - 最初の Zig プログラム
// ============================================================

// ステップ 1：標準ライブラリをインポート
// @import 関数は外部コードを取り込みます
// "std" は便利な関数を持つ Zig の標準ライブラリです
const std = @import("std");

// ステップ 2：main 関数を定義
// - "pub" はパブリック（外部から呼び出し可能）を意味します
// - "fn" は関数を宣言します
// - "main" はプログラムの開始点として認識される特別な名前です
// - "void" はこの関数が何も返さないことを意味します
pub fn main() void {
    // ステップ 3：コンソールに出力
    // std.debug.print() はターミナルにテキストを出力します
    // \n は改行を作成します
    // .{} はフォーマット引数用の空のタプルです
    std.debug.print("Hello, World!\n", .{});
}
```

出力：

```
Hello, World!
```

各部分の詳細

インポート文

```zig
const std = @import("std");
```

この行は 3 つのことを行います：1. `@import("std")` - Zig の標準ライブラリを読み込む 2. `const std` - `std` という名前の定数を作成 3. `=` - ライブラリを定数に代入

次のように考えてください：

```
┌─────────────────────────────────────────────────────────────┐
│                        @import("std")                       │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    標準ライブラリの内容：                                      │
```

```
│    ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│    │   debug      │  │    mem       │  │    fmt       │          │
│    │  .print()    │  │  .copy()     │  │ .format()    │          │
│    └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                   │
│    ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│    │    heap      │  │    fs        │  │    os        │          │
│    │ .allocator()│  │ .openFile()│  │ .system()    │          │
│    └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

main 関数

```
pub fn main() void {
    // ここにコード
}
```

- pub - ランタイムから見えるように関数を公開
- fn - 関数を宣言するキーワード
- main - プログラムのエントリーポイントとして認識される特別な名前
- ( ) - 空の括弧はパラメータがないことを意味
- void - 戻り値の型（何も返さない）
- { } - 中括弧は関数本体を含む

print 文

```
std.debug.print("Hello, World!\n", .{});
```

詳細を見てみましょう：- std - インポートした標準ライブラリ - .debug - デバッグユーティリティ用の std 内の名前空間 - .print() - フォーマットされたテキストを出力する関数 - "Hello, World!\n" - フォーマット文字列（出力するテキスト）- \n - 改行のエスケープシーケンス - .{} - 引数用の空の匿名構造体リテラル（タプル）

変数を使った出力

プログラムを変数を使ってより動的にしましょう：

```
const std = @import("std");

pub fn main() void {
    // 文字列定数を宣言
    // "Zig" は自動的にバイトへのポインタとして型付けされます
    const name = "Zig";

    // 整数定数を宣言
    // : u32 は型を符号なし 32 ビットとして明示的に指定
    const year: u32 = 2016;

    // フォーマット指定子を使って出力：
    // {s} – 文字列としてフォーマット
    // {d} – 10 進数としてフォーマット
    std.debug.print("{s} へようこそ！\n", .{name});
    std.debug.print("{s} は {d} 年に作成されました。\n", .{ name, year });
```

```
    // タプル内の複数の値
    const version = "0.11";
    const is_stable = true;
    std.debug.print(" バージョン：{s}、安定版：{}\n", .{ version, is_stable });
}
```

出力：

```
Zigへようこそ！
Zigは2016年に作成されました。
バージョン：0.11、安定版：true
```

フォーマット指定子リファレンス

フォーマット指定子	
指定子	説明
{s}	文字列
{d}	10進数（整数）
{x}	16進数（小文字）
{X}	16進数（大文字）
{b}	2進数
{c}	文字
{}	デフォルトフォーマット（自動検出）
{any}	任意の型を出力（デバッグ用）
{?}	オプショナル値
{e}	指数表記
修飾子	
{d:5}	最小幅5
{d:0>5}	右寄せ、ゼロ埋め、幅5
{d:<5}	左寄せ、幅5
{d:.2}	小数点以下2桁（浮動小数点用）

高度な出力例

```
const std = @import("std");

pub fn main() void {
    // 数値フォーマット
    const num: i32 = 42;
    const float: f64 = 3.14159265359;
    const byte: u8 = 255;

    std.debug.print("10 進数：     {d}\n", .{num});
    std.debug.print("16 進数：     0x{x}\n", .{byte});      // 0xff
    std.debug.print("2 進数：      0b{b}\n", .{byte});      // 0b11111111
```

```
    std.debug.print(" 浮動小数点：{d:.2}\n", .{float});    // 3.14
    std.debug.print(" 指数表記：  {e}\n", .{float});       // 3.14159e+00

    // パディングと配置
    std.debug.print(" 右寄せ：     |{d:>10}|\n", .{num}); // |        42|
    std.debug.print(" 左寄せ：     |{d:<10}|\n", .{num}); // |42        |
    std.debug.print(" ゼロ埋め：   |{d:0>10}|\n", .{num});// |0000000042|
}
```

出力：

```
10進数：     42
16進数：     0xff
2進数：      0b11111111
浮動小数点：3.14
指数表記：   3.14159265359e0
右寄せ：     |        42|
左寄せ：     |42        |
ゼロ埋め：   |0000000042|
```

キーポイント

1. すべての **Zig** プログラムには **`main`** 関数が必要 - ここから実行が始まります
2. 使用前にインポート - `@import` を使用してライブラリを取り込む
3. フォーマット文字列は型チェックされる - Zig はコンパイル時にフォーマット指定子が引数と一致することを検証
4. セミコロンは必須 - JavaScript とは異なり、省略できません
5. コメントは`//`を使用 - 行の`//`以降はすべて無視されます

第 2 章：変数

変数は値を格納するコンテナです。Zig は変更可能な値と変更不可能な値を明確に区別しています。

2 種類の変数

```
┌──────────────────────────────────────────────────────────┐
│                      Zigの変数型                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────┐      ┌──────────────────┐        │
│   │      const       │      │       var        │        │
│   │   (不変)         │      │    (可変)        │        │
│   ├──────────────────┤      ├──────────────────┤        │
│   │                  │      │                  │        │
│   │ const x = 5;     │      │ var x = 5;       │        │
│   │ x = 10; // エラー! │      │ x = 10; // OK!   │        │
│   │                  │      │                  │        │
│   │ • 変更不可       │      │ • 変更可能       │        │
│   │ • 推奨           │      │ • 必要な時に使用 │        │
│   │ • スレッドセーフ │      │ • 注意が必要     │        │
```

```
│   │  • 最適化可能        │     │                    │      │
│   └──────────────────────┘     └────────────────────┘      │
│                                                             │
│   ルール：変更が必要でない限り常にconstを使用！              │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

定数（const）

定数は初期化後に変更されない値です。Zig ではデフォルトの選択肢です。

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 定数 - 変更されない値
    // ========================================

    // 型は推論される（Zig が判断）
    const greeting = "Hello";           // []const u8（文字列）
    const answer = 42;                   // comptime_int
    const pi = 3.14159;                  // comptime_float

    // 型は明示的（指定する）
    const age: u32 = 25;                 // 符号なし 32 ビット整数
    const temperature: f64 = 98.6;       // 64 ビット浮動小数点
    const is_valid: bool = true;         // ブール値

    std.debug.print("greeting: {s}\n", .{greeting});
    std.debug.print("answer: {d}\n", .{answer});
    std.debug.print("pi: {d:.5}\n", .{pi});
    std.debug.print("age: {d}\n", .{age});
    std.debug.print("temperature: {d}\n", .{temperature});
    std.debug.print("is_valid: {}\n", .{is_valid});

    // これはコンパイルエラーを引き起こします：
    // age = 26;  // エラー：定数に代入できません
}
```

出力：

```
greeting: Hello
answer: 42
pi: 3.14159
age: 25
temperature: 98.6
is_valid: true
```

変数（var）

変数は初期化後に変更できます。必要な場合にのみ使用してください。

```zig
const std = @import("std");

pub fn main() void {
```

```zig
    // ========================================
    // 変数 - 変更可能な値
    // ========================================

    var counter: i32 = 0;
    std.debug.print(" 初期カウンター：{d}\n", .{counter});

    // 変数を変更
    counter = counter + 1;  // 1 になる
    std.debug.print("+1 後：{d}\n", .{counter});

    counter += 10;          // counter = counter + 10 の省略形
    std.debug.print("+=10 後：{d}\n", .{counter});

    counter *= 2;           // 2 倍にする
    std.debug.print("*=2 後：{d}\n", .{counter});

    // var で宣言する変数は型が必要
    var name: []const u8 = "Alice";
    std.debug.print(" 名前：{s}\n", .{name});

    name = "Bob";           // 値を変更
    std.debug.print(" 新しい名前：{s}\n", .{name});
}
```

出力：

```
初期カウンター：0
+1後：1
+=10後：11
*=2後：22
名前：Alice
新しい名前：Bob
```

変数のメモリレイアウト

```
┌──────────────────────────────────────────────────────────┐
│                    メモリレイアウト                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   記述：var x: i32 = 42;                                  │
│                                                          │
│   スタックメモリ：                                         │
│   ┌──────────────────────────────┐                       │
│   │  アドレス    │  値     │  変数      │                  │
│   ├──────────────┼─────────┼────────────┤                  │
│   │  0x1000      │   42    │     x      │ ← 4バイト（i32）  │
│   │  0x1004      │  ...    │   （次）   │                  │
│   └──────────────┴─────────┴────────────┘                  │
│                                                          │
│   異なる型は異なるメモリ量を使用：                          │
│   ┌──────────────────────────────────────────┐            │
│   │ 型       │ サイズ  │ メモリ可視化                     │  │
│
```

```
│    ├──────────┼──────────┼──────────────────────────────┤     │
│    │ u8       │ 1バイト  │ [  ]                         │     │
│    │ u16      │ 2バイト  │ [    ]                       │     │
│    │ u32      │ 4バイト  │ [        ]                   │     │
│    │ u64      │ 8バイト  │ [                ]           │     │
│    │ bool     │ 1バイト  │ [  ]                         │     │
│    └──────────┴──────────┴──────────────────────────────┘     │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

未定義値

変数を宣言して後で値を代入したい場合があります。undefined を使用します：

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // UNDEFINED - 今宣言して、後で代入
    // ========================================

    // 初期化せずに宣言
    // 警告：代入前に未定義値を使用するのは未定義動作！
    var result: i32 = undefined;

    // ... 値を決定するコード ...

    // 値を代入
    result = calculateSomething();

    std.debug.print(" 結果：{d}\n", .{result});
}

fn calculateSomething() i32 {
    return 42;
}
```

出力：

結果：42

キーポイント

1. **var** より **const** を優先 - 不変性はバグを防ぐ
2. 型は推論または明示的 - Zig は可能な場合に型を判断
3. **undefined** は危険 - 読み取り前に代入する場合にのみ使用
4. ブロック式 - 複雑な初期化にはラベル付きブロックを使用
5. コンパイル時評価 - トップレベルの定数は実行時前に計算される

第 3 章：型

Zig は静的型付け言語であり、すべての値はコンパイル時に既知の型を持ちます。型を理解することは、正しい Zig コードを書くための基本です。

型システムの概要

```
┌──────────────────────────────────────────────────────────┐
│                        Zig型階層                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│                          すべての型                       │
│                              │                           │
│                              │                           │
│                              │                           │
│                              │                           │
│        プリミティブ                          複合型        │
│                              │                           │
│                              │                           │
│                              │                           │
│                              │                           │
│   数値型          その他            配列      構造体      共用体 │
│                              │                           │
│                              │                           │
│                              │          スライス    列挙型    オプショナル│
│                              │          ポインタ                 │
│  整数  浮動小数点  ブール    Void      ベクタ                  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

整数型

Zig は整数サイズの細かい制御を提供します：

```
┌──────────────────────────────────────────────────────────┐
│                        整数型                             │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   符号付き（負の値可能）          符号なし（正の値のみ）      │
│   ──────────────────────        ──────────────────────   │
│   i8   : -128 ~ 127             u8   : 0 ~ 255           │
│   i16  : -32768 ~ 32767         u16  : 0 ~ 65535         │
│   i32  : -21億 ~ 21億           u32  : 0 ~ 42億          │
│   i64  : 巨大な範囲             u64  : 0 ~ 巨大          │
│   i128 : さらに巨大             u128 : さらに巨大        │
│                                                          │
│   特殊型：                                                │
│   ────────────                                           │
│   isize : 符号付きポインタサイズ（32または64ビット）        │
│   usize : 符号なしポインタサイズ（配列インデックス用）      │
│                                                          │
│   任意ビット幅：                                          │
│   ──────────────                                         │
│   u1, u2, u3, ..., u65535  （任意のビット幅！）           │
```

```
│   i1, i2, i3, ..., i65535                                    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 符号付き整数（負の値可能）
    // ========================================

    const tiny: i8 = -128;        // 8 ビット：-128 ~ 127
    const small: i16 = -32768;    // 16 ビット
    const medium: i32 = -2147483648; // 32 ビット
    const large: i64 = -9223372036854775808; // 64 ビット

    std.debug.print("i8 最小値：  {d}\n", .{tiny});
    std.debug.print("i16 最小値：{d}\n", .{small});
    std.debug.print("i32 最小値：{d}\n", .{medium});
    std.debug.print("i64 最小値：{d}\n", .{large});

    // ========================================
    // 符号なし整数（正の値のみ）
    // ========================================

    const byte: u8 = 255;          // 8 ビット：0 ~ 255
    const word: u16 = 65535;       // 16 ビット
    const dword: u32 = 4294967295; // 32 ビット
    const qword: u64 = 18446744073709551615; // 64 ビット

    std.debug.print("\nu8 最大値：  {d}\n", .{byte});
    std.debug.print("u16 最大値：{d}\n", .{word});
    std.debug.print("u32 最大値：{d}\n", .{dword});
    std.debug.print("u64 最大値：{d}\n", .{qword});

    // ========================================
    // 任意ビット幅
    // ========================================

    const nibble: u4 = 15;         // 4 ビット：0 ~ 15
    const six_bits: u6 = 63;       // 6 ビット：0 ~ 63
    const twelve: u12 = 4095;      // 12 ビット：0 ~ 4095

    std.debug.print("\nu4 最大値：  {d}\n", .{nibble});
    std.debug.print("u6 最大値：  {d}\n", .{six_bits});
    std.debug.print("u12 最大値：{d}\n", .{twelve});
}
```

出力：

```
i8 最小値：  -128
i16 最小値：-32768
i32 最小値：-2147483648
i64 最小値：-9223372036854775808
```

```
u8 最大値：  255
u16 最大値：65535
u32 最大値：4294967295
u64 最大値：18446744073709551615

u4 最大値：  15
u6 最大値：  63
u12 最大値：4095
```

浮動小数点型

```
┌──────────────────────────────────────────────────────────┐
│                        浮動小数点型                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   型       サイズ      精度          範囲（概算）          │
│   ────     ────       ────────      ──────────────        │
│   f16      16ビット    約3桁         ±65504               │
│   f32      32ビット    約7桁         ±3.4 × 10^38         │
│   f64      64ビット    約15桁        ±1.8 × 10^308        │
│   f128     128ビット   約33桁        ±1.2 × 10^4932       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

ブール型

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // ブール - true または false
    // ========================================

    const is_active: bool = true;
    const is_empty: bool = false;

    std.debug.print("is_active: {}\n", .{is_active});
    std.debug.print("is_empty: {}\n", .{is_empty});

    // 比較からのブール値
    const x: i32 = 10;
    const y: i32 = 20;

    const is_equal = x == y;     // false
    const is_less = x < y;       // true
    const is_greater = x > y;    // false

    std.debug.print("\n{d} == {d}: {}\n", .{ x, y, is_equal });
    std.debug.print("{d} < {d}: {}\n", .{ x, y, is_less });
    std.debug.print("{d} > {d}: {}\n", .{ x, y, is_greater });

    // メモリ内のブール値は 1 バイト
```

```
    std.debug.print("\nbool のサイズ：{d} バイト\n", .{@sizeOf(bool)});
}
```

出力：

```
is_active: true
is_empty: false

10 == 20: false
10 < 20: true
10 > 20: false

boolのサイズ：1バイト
```

キーポイント

1. 適切なサイズを選択 - i8 で十分な場合に i64 を使用しない
2. 符号付き **vs** 符号なし - カウント/インデックスには符号なし、算術には符号付き
3. 浮動小数点の精度 - f64 はより正確だがメモリを多く使用
4. 明示的なキャスト - Zig は縮小時に明示的なキャストを要求
5. **comptime** 型 - 型なしの数値リテラルは `comptime_int` または `comptime_float`

第 4 章：演算子

演算子は値に対して操作を行う記号です。Zig は算術、比較、論理、ビット操作のための包括的な演算子セットを提供します。

演算子カテゴリ

```
┌──────────────────────────────────────────────────────────────┐
│                      演算子カテゴリ                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  算術              比較              論理                    │
│  ──────────       ──────────        ───────                 │
│  +  加算           ==  等しい        and  論理AND            │
│  -  減算           !=  等しくない     or   論理OR            │
│  *  乗算           <   より小さい     !    論理NOT           │
│  /  除算           >   より大きい                             │
│  %  剰余           <=  以下                                  │
│  -  否定           >=  以上                                  │
│                                                              │
│  ビット演算         代入              特殊                    │
│  ───────          ──────────        ───────                 │
│  &   AND          =   代入          ++  (Zigにはない！)      │
│  |   OR           +=  加算代入       --  (Zigにはない！)      │
│  ^   XOR          -=  減算代入       ?:  (if-elseを使用)      │
│  ~   NOT          *=  乗算代入                               │
│  <<  左シフト      /=  除算代入                               │
│  >>  右シフト      %=  剰余代入                               │
```

```
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

算術演算子

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 算術演算子 ===\n\n", .{});

    const a: i32 = 17;
    const b: i32 = 5;

    // 基本操作
    std.debug.print("{d} + {d} = {d}\n", .{ a, b, a + b });   // 22
    std.debug.print("{d} - {d} = {d}\n", .{ a, b, a - b });   // 12
    std.debug.print("{d} * {d} = {d}\n", .{ a, b, a * b });   // 85

    // Zig での除算は明示的な関数を必要とする
    std.debug.print("{d} / {d} = {d} (切り捨て)\n", .{ a, b, @divTrunc(a, b) });

    // 剰余
    std.debug.print("{d} %% {d} = {d}\n", .{ a, b, @mod(a, b) });

    // 否定
    std.debug.print("-{d} = {d}\n", .{ a, -a });
}
```

出力：

```
=== 算術演算子 ===

17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 (切り捨て)
17 % 5 = 2
-17 = -17
```

キーポイント

1. ++ や–演算子はない - += 1 と–= 1 を代わりに使用
2. 明示的なオーバーフロー処理 - ラップには +%、–%、*%、飽和には +|、–|、*|
3. 短絡評価 - and と or は必要なければ 2 番目のオペランドを評価しない
4. ビット演算子 - フラグ、マスク、低レベルプログラミングに強力
5. 除算関数 - 明示的な動作のために @divTrunc、@divFloor、@mod、@rem を使用

第 5 章：配列

配列は同じ型の要素の固定サイズで連続したコレクションです。Zig で最も基本的なデータ構造の 1 つです。

配列のメモリレイアウト

```
┌──────────────────────────────────────────────────────────────┐
│                          メモリ内の配列                          │
├──────────────────────────────────────────────────────────────┤
│                                                                │
│   宣言：const arr: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };      │
│                                                                │
│   メモリレイアウト（4バイトi32を想定）：                          │
│                                                                │
│   アドレス：0x100   0x104   0x108   0x10C   0x110              │
│                                                                │
│   値：      ┌─────┬─────┬─────┬─────┬─────┐                    │
│             │ 10  │ 20  │ 30  │ 40  │ 50  │                    │
│             └─────┴─────┴─────┴─────┴─────┘                    │
│   インデックス：[0]    [1]    [2]    [3]    [4]                 │
│                                                                │
│   主要な特性：                                                  │
│   • サイズはコンパイル時に固定                                   │
│   • 要素は連続（隙間なし）                                       │
│   • インデックスによるアクセスはO(1)                             │
│   • 合計サイズ = 要素サイズ × 個数 = 4 × 5 = 20バイト            │
│                                                                │
└──────────────────────────────────────────────────────────────┘
```

配列の作成

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 配列の作成 ===\n\n", .{});

    // 方法 1：明示的な型と値
    const explicit: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };
    std.debug.print(" 明示的：{any}\n", .{explicit});

    // 方法 2：[_] で長さを推論
    const inferred = [_]i32{ 1, 2, 3, 4 };  // コンパイラがカウント：4 要素
    std.debug.print(" 推論：{any}\n", .{inferred});
    std.debug.print(" 長さ：{d}\n", .{inferred.len});

    // 方法 3：同じ値で埋める
    const zeros = [_]i32{0} ** 5;    // [0, 0, 0, 0, 0]
    const ones = [_]i32{1} ** 3;     // [1, 1, 1]

    std.debug.print(" ゼロ：{any}\n", .{zeros});
    std.debug.print("1：{any}\n", .{ones});

    // 方法 4：パターンを繰り返す
    const pattern = [_]i32{ 1, 2 } ** 3;  // [1, 2, 1, 2, 1, 2]
    std.debug.print(" パターン：{any}\n", .{pattern});
}
```

出力：

```
=== 配列の作成 ===

明示的：{ 10, 20, 30, 40, 50 }
推論：{ 1, 2, 3, 4 }
長さ：4
ゼロ：{ 0, 0, 0, 0, 0 }
1：{ 1, 1, 1 }
パターン：{ 1, 2, 1, 2, 1, 2 }
```

キーポイント

1. 固定サイズ - 配列の長さはコンパイル時に既知で変更不可
2. ゼロインデックス - 最初の要素はインデックス 0
3. 境界チェック - Zig はデバッグモードで配列境界をチェック
4. 連続メモリ - 要素は連続して格納される
5. 推論長には `[_]` を使用 - コンパイラが要素をカウント
6. `++` と `**` はコンパイル時のみ - 実行時に配列を連結できない

第 6 章：スライス

スライスは配列や他の連続メモリへのビューです。配列とは異なり、スライスはデータを所有せず、参照するだけです。

スライス vs 配列

```
┌──────────────────────────────────────────────────────────────┐
│                        配列 vs スライス                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   配列：[5]i32                                                 │
│   • 固定サイズ（5要素）                                          │
│   • サイズはコンパイル時に既知                                    │
│   • データを所有                                                 │
│   • インラインで格納                                             │
│                                                              │
│   ┌─────┬─────┬─────┬─────┬─────┐                              │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │  （インラインデータ）          │
│   └─────┴─────┴─────┴─────┴─────┘                              │
│                                                              │
│   スライス：[]i32                                               │
│   • 可変サイズ                                                   │
│   • サイズは実行時に既知                                          │
│   • データを参照（所有しない）                                     │
│   • ポインタ + 長さのみ                                          │
│                                                              │
│   ┌──────────────┐                                             │
│   │ ptr │ len=5  │──────┐                                      │
│   └──────────────┘      │                                      │
```

```
│                              ▼                                  │
│                                                                 │
│    ┌─────┬─────┬─────┬─────┬─────┐   （別の場所のデータ）      │
│    │ 10  │ 20  │ 30  │ 40  │ 50  │                             │
│    └─────┴─────┴─────┴─────┴─────┘                             │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

スライスの作成

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== スライスの作成 ===\n\n", .{});

    const array = [_]i32{ 10, 20, 30, 40, 50, 60, 70 };

    // 方法 1：配列全体をスライス
    const all: []const i32 = &array;
    std.debug.print(" 全体：{any}\n", .{all});
    std.debug.print(" 長さ：{d}\n", .{all.len});

    // 方法 2：範囲でスライス [start..end)
    const middle = array[2..5];  // インデックス 2, 3, 4 の要素
    std.debug.print("\nmiddle[2..5]: {any}\n", .{middle});

    // 方法 3：開いたスライス
    const from_start = array[0..3];   // 最初の 3 要素
    const to_end = array[4..];        // インデックス 4 から終わりまで

    std.debug.print("array[0..3]: {any}\n", .{from_start});
    std.debug.print("array[4..]:  {any}\n", .{to_end});
}
```

出力：

```
=== スライスの作成 ===

全体：{ 10, 20, 30, 40, 50, 60, 70 }
長さ：7

middle[2..5]: { 30, 40, 50 }
array[0..3]: { 10, 20, 30 }
array[4..]:  { 50, 60, 70 }
```

キーポイント

1. スライスはデータを所有しない - 既存メモリへのビュー
2. 範囲構文 **[start..end]** - 終了は排他的
3. 柔軟な関数パラメータ - []const T を使用して任意のサイズを受け入れる
4. 実行時の長さ - 配列と異なり、スライスの長さは実行時に計算可能
5. メモリ効率的 - ポインタ + 長さのみ（64 ビットで 16 バイト）

第 7 章：ポインタ

ポインタはメモリアドレスを格納します。システムプログラミングに不可欠で、コピーせずにデータを参照できます。

ポインタの基本

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== ポインタの基本 ===\n\n", .{});

    // 変数を作成
    var value: i32 = 42;

    // & でポインタを取得
    const ptr: *i32 = &value;

    // .* でポインタ経由で読み取り
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});

    // ポインタ経由で変更
    ptr.* = 100;
    std.debug.print("\nptr.* = 100 の後：\n", .{});
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});
}
```

出力：

```
=== ポインタの基本 ===

value = 42
ptr.* = 42

ptr.* = 100の後：
value = 100
ptr.* = 100
```

キーポイント

1. **&** はアドレスを取得 - &x で x へのポインタを取得
2. **.*** は参照解除 - ptr.* で値にアクセス
3. ポインタは変更を可能に - ポインタ渡しで元データを変更
4. **const** ポインタ - *const T はそのポインタ経由での変更を防ぐ
5. デフォルトで **null** なし - 通常のポインタは null にできない；null 可能には ?*T を使用

第 8 章：文字列

Zig では、文字列は単なるバイトの配列（[]const u8）です。特別な文字列型はありません - これにより言語はシンプルになり、完全な制御が得られます。

文字列型

```
┌──────────────────────────────────────────────────────────────┐
│                      Zigの文字列型                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   型                    説明                                 │
│   ────                  ──────────                           │
│   []const u8            バイトのスライス（最も一般的）        │
│   *const [N:0]u8        null終端文字列へのポインタ            │
│   [:0]const u8          センチネル終端スライス                │
│   [N]u8                 固定サイズバイト配列                  │
│                                                              │
│   重要な洞察：Zigでは文字列は特別ではない。                    │
│   単なるu8（バイト）の配列です。                               │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 文字列の基本 ===\n\n", .{});

    const greeting: []const u8 = "Hello, Zig!";

    std.debug.print(" 文字列：{s}\n", .{greeting});
    std.debug.print(" 長さ：{d}\n", .{greeting.len});

    // 個々のバイトにアクセス
    std.debug.print(" 最初のバイト：'{c}' (ASCII {d})\n", .{ greeting[0], greeting[0] });

    // 文字を反復
    std.debug.print("\n文字：", .{});
    for (greeting) |char| {
        std.debug.print("{c} ", .{char});
    }
    std.debug.print("\n", .{});
}
```

出力：

```
=== 文字列の基本 ===

文字列：Hello, Zig!
長さ：11
最初のバイト：'H' (ASCII 72)

文字：H e l l o ,  Z i g !
```

キーポイント

1. 文字列は **`[]const u8`** - 単なるバイトスライス、特別なものではない
2. 操作には **`std.mem`** を使用 - `eql`、`indexOf`、`startsWith` など
3. **`++`** 連結はコンパイル時のみ - 実行時に連結できない
4. デフォルトで **UTF-8** - Zig 文字列は UTF-8 エンコード
5. **`{s}`** フォーマット指定子 - 文字列出力に使用

第 9 章：構造体

構造体は関連するデータをカスタム型にグループ化できます。コードを整理するための最も重要な機能の 1 つです。

構造体の作成

```zig
const std = @import("std");

// 構造体型を定義
const Point = struct {
    x: i32,
    y: i32,
};

// デフォルト値を持つ構造体
const Rectangle = struct {
    x: i32 = 0,
    y: i32 = 0,
    width: u32 = 100,
    height: u32 = 100,
};

pub fn main() void {
    std.debug.print("=== 構造体の作成 ===\n\n", .{});

    // すべてのフィールドで作成
    const p1 = Point{ .x = 10, .y = 20 };
    std.debug.print("p1: ({d}, {d})\n", .{ p1.x, p1.y });

    // デフォルトで作成
    const r1 = Rectangle{};  // すべてデフォルト
    std.debug.print("r1: pos=({d},{d}) size={d}x{d}\n", .{ r1.x, r1.y, r1.width, r1.height });

    // 一部のデフォルトを上書き
    const r2 = Rectangle{ .x = 50, .y = 50, .width = 200 };
    std.debug.print("r2: pos=({d},{d}) size={d}x{d}\n", .{ r2.x, r2.y, r2.width, r2.height });
}
```

出力：

```
=== 構造体の作成 ===

p1: (10, 20)
```

```
r1: pos=(0,0) size=100x100
r2: pos=(50,50) size=200x100
```

キーポイント

1. 名前付きフィールド - `.fieldname` でアクセス
2. デフォルト値 - フィールドはデフォルトを持てる
3. メソッド - `self` パラメータを持つ関数
4. 関連関数 - `self` なしの関数 (static のような)
5. ポインタメソッド - 構造体を変更するには `*Self` を使用

第 10 章：列挙型

列挙型は固定された名前付き値のセットを持つ型を定義します。選択や状態を表すのに最適です。

```
const std = @import("std");

const Color = enum {
    red,
    green,
    blue,
    yellow,
    purple,
};

const Direction = enum {
    north,
    south,
    east,
    west,

    // 列挙型はメソッドを持てる！
    pub fn opposite(self: Direction) Direction {
        return switch (self) {
            .north => .south,
            .south => .north,
            .east => .west,
            .west => .east,
        };
    }
};

pub fn main() void {
    std.debug.print("=== 列挙型の基本 ===\n\n", .{});

    const favorite: Color = .blue;
    std.debug.print(" お気に入り：{}\n", .{favorite});

    // switch で列挙型を使用 ( すべてのケースを処理する必要がある )
    const message = switch (favorite) {
        .red => " 止まれ！",
        .green => " 進め！",
```

```
        .blue => " クール",
        .yellow => " 注意",
        .purple => " 王族",
    };
    std.debug.print(" メッセージ：{s}\n", .{message});

    // メソッド
    const dir: Direction = .north;
    std.debug.print("\n方向：{}\n", .{dir});
    std.debug.print(" 反対：{}\n", .{dir.opposite()});
}
```

出力：

```
=== 列挙型の基本 ===

お気に入り：Color.blue
メッセージ：クール

方向：Direction.north
反対：Direction.south
```

第 11 章：共用体

共用体は複数の型のいずれかを保持できますが、一度に 1 つだけです。タグ付き共用体はどのバリアントがアクティブか知っています。

```
const std = @import("std");

const Value = union(enum) {
    int: i64,
    float: f64,
    text: []const u8,
    none: void,

    pub fn describe(self: Value) void {
        switch (self) {
            .int => |n| std.debug.print(" 整数：{d}\n", .{n}),
            .float => |f| std.debug.print(" 浮動小数点：{d:.2}\n", .{f}),
            .text => |s| std.debug.print(" テキスト：{s}\n", .{s}),
            .none => std.debug.print(" なし\n", .{}),
        }
    }
};

pub fn main() void {
    std.debug.print("=== タグ付き共用体 ===\n\n", .{});

    const values = [_]Value{
        .{ .int = 42 },
        .{ .float = 3.14159 },
        .{ .text = "Hello" },
```

```
        .none,
    };

    for (values) |v| {
        v.describe();
    }
}
```

出力：

```
=== タグ付き共用体 ===

整数：42
浮動小数点：3.14
テキスト：Hello
なし
```

第 12 章：制御フロー

Zig は標準的な制御フローを提供します：if、switch、while、for。それぞれ式として使用できます。

if 式

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== IF 式 ===\n\n", .{});

    const x: i32 = 42;

    // 基本的な if 文
    if (x > 0) {
        std.debug.print("x は正\n", .{});
    }

    // if-else
    if (x % 2 == 0) {
        std.debug.print("x は偶数\n", .{});
    } else {
        std.debug.print("x は奇数\n", .{});
    }

    // 式としての if（三項演算子のような）
    const abs_x = if (x >= 0) x else -x;
    std.debug.print("{d} の絶対値：{d}\n", .{ x, abs_x });
}
```

ループ

```zig
const std = @import("std");

pub fn main() void {
    // while ループ
    std.debug.print("=== WHILE ループ ===\n", .{});

    var i: u32 = 0;
    while (i < 5) {
        std.debug.print("{d} ", .{i});
        i += 1;
    }
    std.debug.print("\n", .{});

    // for ループ
    std.debug.print("\n=== FOR ループ ===\n", .{});

    const numbers = [_]i32{ 10, 20, 30, 40, 50 };

    for (numbers) |n| {
        std.debug.print("{d} ", .{n});
    }
    std.debug.print("\n", .{});

    // インデックス付き
    for (numbers, 0..) |n, idx| {
        std.debug.print("[{d}]={d} ", .{ idx, n });
    }
    std.debug.print("\n", .{});
}
```

第 13 章：関数

関数は再利用可能なコードブロックです。Zig 関数はコンパイル時パラメータを持つことができ、非常に柔軟です。

```zig
const std = @import("std");

// シンプルな関数
fn add(a: i32, b: i32) i32 {
    return a + b;
}

// void 戻り値 - 戻り値なし
fn greet(name: []const u8) void {
    std.debug.print(" こんにちは、{s}！\n", .{name});
}

// 再帰関数
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
```

```
}

pub fn main() void {
    std.debug.print("=== 関数 ===\n\n", .{});

    const sum = add(5, 3);
    std.debug.print("5 + 3 = {d}\n", .{sum});

    greet("Zig");

    std.debug.print("5! = {d}\n", .{factorial(5)});
    std.debug.print("10! = {d}\n", .{factorial(10)});
}
```

出力：

```
=== 関数 ===

5 + 3 = 8
こんにちは、Zig！
5! = 120
10! = 3628800
```

第 14 章：エラー処理

Zig にはエラー共用体による組み込みエラー処理があります。エラーは例外ではなく値です。

```
const std = @import("std");

// エラーセットを定義
const MathError = error{
    DivisionByZero,
    NegativeNumber,
};

// 失敗する可能性のある関数
fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

pub fn main() void {
    std.debug.print("=== エラー処理 ===\n\n", .{});

    // 方法 1：if でチェック
    if (divide(10, 2)) |result| {
        std.debug.print("10 / 2 = {d}\n", .{result});
    } else |err| {
        std.debug.print(" エラー：{}\n", .{err});
    }

    // 方法 2：catch でデフォルト
```

```
    const safe = divide(10, 0) catch 0;
    std.debug.print("10 / 0 ( デフォルト付き ): {d}\n", .{safe});
}
```

出力：

```
=== エラー処理 ===

10 / 2 = 5
10 / 0 ( デフォルト付き ) : 0
```

第 15 章：オプショナル

オプショナルは存在しない可能性のある値を表します。null ポインタより安全です。

```
const std = @import("std");

fn find(haystack: []const u8, needle: u8) ?usize {
    for (haystack, 0..) |byte, index| {
        if (byte == needle) return index;
    }
    return null;
}

pub fn main() void {
    std.debug.print("=== オプショナル ===\n\n", .{});

    const text = "Hello, World!";
    if (find(text, 'W')) |index| {
        std.debug.print("'W'はインデックス {d} で見つかりました\n", .{index});
    }

    if (find(text, 'Z')) |index| {
        std.debug.print("'Z'はインデックス {d} で見つかりました\n", .{index});
    } else {
        std.debug.print("'Z'は見つかりませんでした\n", .{});
    }

    // orelse - デフォルトを提供
    const val1: ?i32 = 42;
    const val2: ?i32 = null;

    std.debug.print("\nval1 orelse 0: {d}\n", .{val1 orelse 0});
    std.debug.print("val2 orelse -1: {d}\n", .{val2 orelse -1});
}
```

出力：

```
=== オプショナル ===

'W'はインデックス7で見つかりました
'Z'は見つかりませんでした
```

```
val1 orelse 0: 42
val2 orelse -1: -1
```

第 16 章：Comptime

Comptime はコンパイル時にコードを実行します。マクロなしで強力なメタプログラミングを可能にします。

```
const std = @import("std");

const KILOBYTE = 1024;
const MEGABYTE = KILOBYTE * 1024;
const GIGABYTE = MEGABYTE * 1024;

fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

const FACT_10 = factorial(10);  // コンパイル時に計算！

pub fn main() void {
    std.debug.print("=== COMPTIME ===\n\n", .{});

    std.debug.print("1 KB = {d} バイト\n", .{KILOBYTE});
    std.debug.print("1 MB = {d} バイト\n", .{MEGABYTE});
    std.debug.print("1 GB = {d} バイト\n", .{GIGABYTE});

    std.debug.print("\n10! = {d}（コンパイル時に計算！）\n", .{FACT_10});
}
```

第 17 章：ジェネリクス

ジェネリクスは任意の型で動作するコードを書くことができます。

```
const std = @import("std");

// ジェネリック関数
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn swap(comptime T: type, a: *T, b: *T) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
}

pub fn main() void {
    std.debug.print("=== ジェネリクス ===\n\n", .{});
```

```
    std.debug.print("max(i32, 5, 10) = {d}\n", .{max(i32, 5, 10)});
    std.debug.print("max(f64, 3.14, 2.71) = {d}\n", .{max(f64, 3.14, 2.71)});

    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\nスワップ前：x={d}, y={d}\n", .{ x, y });
    swap(i32, &x, &y);
    std.debug.print(" スワップ後：x={d}, y={d}\n", .{ x, y });
}
```

第 18 章：メモリ管理

Zig はメモリ割り当ての明示的な制御を提供します。

```
const std = @import("std");

pub fn main() !void {
    std.debug.print("=== メモリ管理 ===\n\n", .{});

    // スタックメモリ（自動）
    var stack_var: i32 = 42;
    std.debug.print(" スタック変数：{d}\n", .{stack_var});

    // ヒープメモリ（手動）
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // 単一の値を割り当て
    const ptr = try allocator.create(i32);
    defer allocator.destroy(ptr);
    ptr.* = 100;
    std.debug.print("\nヒープ値：{d}\n", .{ptr.*});

    // スライスを割り当て
    const slice = try allocator.alloc(i32, 5);
    defer allocator.free(slice);
    for (slice, 0..) |*item, i| {
        item.* = @as(i32, @intCast((i + 1) * 10));
    }
    std.debug.print(" ヒープスライス：{any}\n", .{slice});
}
```

第 19 章：テスト

Zig には組み込みのテストサポートがあります。

```
const std = @import("std");
const expect = std.testing.expect;
```

```zig
const expectEqual = std.testing.expectEqual;

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn factorial(n: u32) u32 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

test " 基本的な加算" {
    try expectEqual(@as(i32, 5), add(2, 3));
    try expectEqual(@as(i32, 0), add(-5, 5));
}

test " 階乗" {
    try expectEqual(@as(u32, 1), factorial(0));
    try expectEqual(@as(u32, 120), factorial(5));
}

pub fn main() void {
    std.debug.print(" テストを実行：zig test filename.zig\n", .{});
}
```

第 20 章：C 相互運用

Zig は C コードを直接呼び出すことができます。

```zig
const std = @import("std");
const c = @cImport({
    @cInclude("math.h");
});

pub fn main() void {
    std.debug.print("=== C 相互運用 ===\n\n", .{});

    const x: c.double = 2.0;

    std.debug.print("sqrt({d}) = {d}\n", .{ x, c.sqrt(x) });
    std.debug.print("pow({d}, 3) = {d}\n", .{ x, c.pow(x, 3.0) });
}

// C から呼び出すための関数をエクスポート
export fn zig_add(a: c_int, b: c_int) c_int {
    return a + b;
}
```

付録 A：Zig チートシート

```
┌──────────────────────────────────────────────────────────┐
│                        ZIGチートシート                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  変数                                                    │
│  ────────────────────────────────────────────────────    │
│  const x: i32 = 42;        // 不変                       │
│  var y: i32 = 0;           // 可変                       │
│  var z: i32 = undefined;   // 未初期化                    │
│                                                          │
│  型                                                      │
│  ────────────────────────────────────────────────────    │
│  i8, i16, i32, i64, i128   // 符号付き整数                 │
│  u8, u16, u32, u64, u128   // 符号なし整数                 │
│  f16, f32, f64, f128       // 浮動小数点                 │
│  bool                      // ブール                     │
│  []T                       // スライス                   │
│  [N]T                      // 配列                       │
│  *T                        // ポインタ                   │
│  ?T                        // オプショナル               │
│  !T または E!T              // エラー共用体               │
│                                                          │
│  制御フロー                                              │
│  ────────────────────────────────────────────────────    │
│  if (cond) { } else { }                                  │
│  switch (x) { val => {}, else => {} }                    │
│  while (cond) : (cont) { }                               │
│  for (slice) |item| { }                                  │
│  for (slice, 0..) |item, i| { }                          │
│                                                          │
│  エラー処理                                              │
│  ────────────────────────────────────────────────────    │
│  try expression           // エラーを伝播                 │
│  catch |err| { }          // エラーを処理                 │
│  orelse default           // オプショナルをアンラップ     │
│  if (opt) |val| { }       // オプショナルをアンラップ     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

著者について

JW Lee はシステムプログラミングに情熱を持ち、複雑なトピックを初心者にも分かりやすくすることに取り組むソフトウェア開発者です。この本は、Zig プログラミング言語への最も包括的でありながらアクセスしやすい入門書を作成するための彼の努力を表しています。

やさしい *Zig - Zig* プログラミング言語の包括的な入門ガイド

Copyright 2024 JW Lee

この本のすべての例は *MIT* ライセンスの下で提供されています。